

[VJC/2025/Q3]

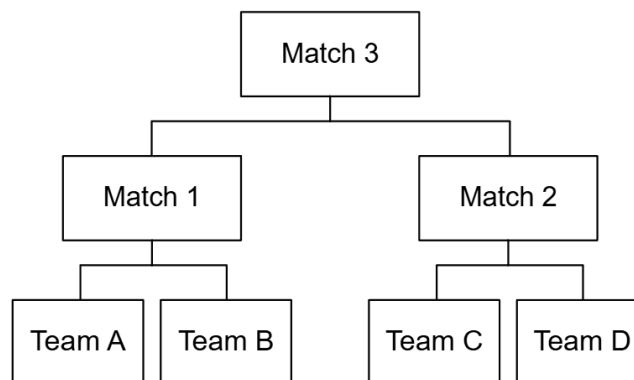
3 Name your Jupyter Notebook as:

TASK3_<your name>_<class>_<index number>.ipynb

Write a program to represent the matches in a sports tournament as a binary tree where:

- each leaf node represents a participating team
- each internal node represents a match between two winners from lower rounds
- the root node represents the final match (champion).

The number of teams must be an exponential growth with base 2, i.e. 2^n where n is a positive integer, so that every round has equal matches until the final match. An example of a binary tree used to represent matches in the sports tournament involving 4 participating teams is as follows:



The program uses object-oriented programming to implement the binary tree and its nodes.

The class `Node` contains three attributes:

- `team`: name of the team or match label stored at the node
- `left_ptr`: pointer to the left child of the node, `None` if there is no left child
- `right_ptr`: pointer to the right child of the node, `None` if there is no right child

The class `Node` has a constructor that initialises the three attributes.

The class `TournamentTree` is a binary tree that represents the matches in the tournament and contains one attribute:

- `root`: pointer to the root node of the tree, `None` if the tree is empty.

The class `TournamentTree` has the following methods:

- a constructor that initialises the `root` pointer to an appropriate value
- `create_tree(num_of_teams)` that takes the number of teams, `num_of_teams`, as a parameter and create the binary tree representing the matches.
- `level_order()` that displays the names of the matches or teams by levels of the binary tree
- `pre_order()` that displays the names of the matches or teams by performing pre-order traversal on the binary tree
- `post_order()` that displays the names of the matches or teams by performing post-order traversal on the binary tree
- `search(target_name)` that takes the name of the team or match to be searched for in the binary tree. It returns `True` if the name can be found, or `False` otherwise
- `calculate_matches(num_of_teams)` that takes the number of teams, `num_of_teams`, participating in the sports tournament and returns the number of matches needed.

[VJC/2025/Q3 Cont ']

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: 

#Task 3.1  
Program code


```

Output:

Task 3.1

Write program code to declare the class `Node` and its constructor. [2]

Task 3.2

Write program code to declare the class `TournamentTree` and its constructor. [2]

Task 3.3

Amend your code for **Task 3.2** to declare the `TournamentTree` method `create_tree()` and add in the code provided for `level_order()` from the file `level_order.py`.

In a new cell, write program code to

- initialise a new `TournamentTree`
- call `create_tree()` to create a tree with 8 participating teams, named Team A - H
- call `level_order()` to display the tree by level. [8]

Test your program and show the output. [1]

Task 3.4

Amend your code for **Task 3.2** to declare the `TournamentTree` methods `pre_order()` and `post_order()`.

In a new cell, write program code to

- call `pre_order()` to perform pre-order traversal
- call `post_order()` to perform post-order traversal. [4]

Test your program and show the output. [2]

Task 3.5

Amend your code for **Task 3.2** to declare the `TournamentTree` method `search()`.

In a new cell, write program code to test the `search()` method using these 2 test cases,

- Team E
- Team J [3]

Show the output. [1]

Save your Jupyter notebook for Task 3.